SASS

Tous droits réservés à STUDI - Reproduction interdite

Table des matières

I - Contexte 3

II - Principales fonctions de SASS 4

A. Principales fonctions de SASS 4

B. Exercice : Quiz 8

III - Techniques avancées SASS : optimiser votre développement CSS 10

A. Techniques avancées SASS : optimiser votre développement CSS 10

1. Compiler du code SASS en CSS 10
2. Les opérateurs mathématiques dans SASS 12
3. Les boucles dans SASS 13
4. Les fonctions dans SASS 14

B. Exercice : Quiz 16

IV - Des ressources adéquates pour un développement efficace du SASS 18

A. Des ressources adéquates pour un développement efficace du SASS 18

1. Vue d'ensemble des frameworks et bibliothèques SASS populaires qui peuvent être utilisés pour optimiser le développement 18
2. La documentation de SASS 18

B. Exercice : Quiz 20

V - Synthèse 22

VI - Auto-évaluation 24

A. Exercice 24

B. Test 25

Solutions des exercices 27

Tous droits réservés à STUDI - Reproduction interdite

I Contexte

CSS (*Cascading Style Sheets*) est un langage de feuille de style largement utilisé pour décrire la présentation des pages web. Toutefois, l'écriture des feuilles de style CSS peut s'avérer fastidieuse et chronophage, en particulier lorsqu'il s'agit de maintenir des bases de code vastes et complexes. En parallèle, SASS (*Syntactically Awesome Style Sheets*) est un préprocesseur CSS qui offre aux développeurs un ensemble d'outils puissants pour écrire un code plus propre et plus efficace.

SASS étend les capacités de CSS avec des fonctionnalités telles que les variables, l'imbrication et les mixins. Avec SASS, les développeurs peuvent écrire un code plus facile à lire, plus rapide à maintenir et plus flexible.

Au fil de ce cours, nous explorerons les principales fonctions de SASS, son fonctionnement global et les ressources que vous pouvez utiliser pour exploiter tout son potentiel. Vous aurez ainsi une solide compréhension du fonctionnement de SASS, ainsi que les outils et techniques dont vous avez besoin pour l'utiliser efficacement dans vos propres projets.

Tous droits réservés à STUDI - Reproduction interdite

II Principales fonctions de SASS

A. Principales fonctions de SASS

Un langage préprocesseur

SASS est donc ce qu'on appelle un langage préprocesseur. En réalité, on peut voir SASS comme un « macro-langage », qui encapsule le langage CSS.

Lorsqu'on va compiler notre langage SASS, c'est en fait du code CSS qui va être généré, car les navigateurs ne peuvent pas interpréter directement du langage SASS.

Les variables SASS

Les variables SASS sont une fonctionnalité clé du préprocesseur qui permet aux développeurs de simplifier leur code et de le rendre plus facile à mettre à jour et à maintenir. Les variables permettent aux développeurs de définir des valeurs une seule fois, puis de les réutiliser dans leur feuille de style.

Exemple

Voici un exemple de CSS classique :

```css
/* Avec du pur CSS */
h1 {
  color: #0077cc;
  font-size: 32px;
  line-height: 40px;
  margin-bottom: 24px;
}

h2 {
  color: #0077cc;
  font-size: 24px;
  line-height: 32px;
  margin-bottom: 16px;
}
```

Dans cet exemple, nous avons défini la couleur de la marque comme étant #0077cc et l'avons utilisée à plusieurs endroits. Cependant, nous avons également dû répéter d'autres valeurs, telles que la taille de la police, l'épaisseur des lignes et la marge, ce qui peut rendre notre code moins facile à maintenir.

Exemple

Voyons un exemple utilisant les variables SASS :

```scss
/* Avec SASS */
$brand-color: #0077cc;
$heading-font-size: 32px;
$heading-line-height: 40px;
$heading-margin-bottom: 24px;

h1 {
  color: $brand-color;
  font-size: $heading-font-size;
  line-height: $heading-line-height;
```


Tous droits réservés à STUDI - Reproduction interdite

```
  margin-bottom: $heading-margin-bottom;
}

h2 {
  color: $brand-color;
  font-size: 24px;
  line-height: 32px;
  margin-bottom: 16px;
}
```

Dans cet exemple, nous avons défini des variables pour toutes nos valeurs répétées et les avons ensuite utilisées dans notre feuille de style. Non seulement notre code est plus facile à maintenir, mais il est également plus facile de mettre à jour notre conception à l'avenir. À noter que le CSS permet aussi de créer des variables pour les couleurs, mais de façon plus limitée, et notamment moins compatible avec les navigateurs.

Dans l'ensemble, l'utilisation des variables SASS peut grandement simplifier le développement CSS et améliorer la maintenabilité de notre code. En définissant des valeurs une seule fois et en les réutilisant dans notre feuille de style, nous pouvons gagner du temps et réduire le risque d'erreurs.

Il y a de nombreux types de variables : des nombres, des chaînes de caractères, des listes, dictionnaires, etc. Dans ce cours, nous verrons différentes syntaxes pour ces types-ci.

L'imbrication dans SASS

SASS introduit également le concept d'imbrication, qui permet aux développeurs de regrouper des styles apparentés dans une structure hiérarchique. Cela peut rendre le code plus lisible et plus facile à comprendre, en particulier pour les feuilles de style volumineuses ou complexes.

Exemple

Pour utiliser l'imbrication dans SASS, il suffit d'inclure un sélecteur à l'intérieur d'un autre, en le séparant par un espace. Par exemple, pour styliser tous les liens d'un menu de navigation, vous pouvez écrire :

```
/* Avec CSS */
nav ul li a {
  color: #333;
  text-decoration: none;
}

/* Avec SASS */
nav {
  ul {
    li {
      a {
        color: #333;
        text-decoration: none;
      }
    }
  }
}
```

Tous droits réservés à STUDI - Reproduction interdite

En imbriquant les sélecteurs de cette manière, nous pouvons clairement voir la structure de notre code et la manière dont les différents éléments sont liés. Il est ainsi plus facile de naviguer dans notre code et d'apporter des modifications à des éléments spécifiques.

L'imbrication peut également être utilisée en conjonction avec les variables SASS et d'autres fonctionnalités pour créer un code plus efficace et plus facile à maintenir. Par exemple, vous pouvez utiliser un bloc imbriqué pour définir un ensemble de styles pour une section spécifique de votre site web, puis utiliser des variables pour définir des valeurs spécifiques à cette section. Vous éviterez ainsi de dupliquer le code et pourrez plus facilement mettre à jour votre conception à l'avenir.

Exemple

```scss
/* Utiliser l'imbrication et les variables dans SASS */
$primary-color: #1E88E5;

header {
  background-color: $primary-color;

  nav {
    ul {
      li {
        a {
          color: $primary-color;
        }
      }
    }
  }
}

section {
  background-color: $primary-color;

  h2 {
    color: white;
  }
}
```

Dans cet exemple, nous utilisons une variable appelée `$primary-color` pour définir la palette de couleurs principale de notre site web. En définissant la variable en haut de notre feuille de style, nous pouvons facilement changer la couleur de l'ensemble de notre site web, en mettant à jour la variable en un seul endroit.

Nous utilisons ensuite l'imbrication SASS pour définir plusieurs ensembles de styles pour les différentes parties de notre site web. Dans la section de l'en-tête, nous définissons la couleur d'arrière-plan sur notre couleur primaire, puis nous utilisons l'imbrication pour styliser les liens à l'intérieur du menu de navigation avec la même couleur.

De même, dans le bloc de section, nous définissons la couleur d'arrière-plan sur notre couleur primaire et nous donnons un style blanc au texte de l'en-tête. En utilisant l'imbrication et les variables de cette manière, nous pouvons créer une hiérarchie de styles claire, facile à lire et à maintenir.

Tous droits réservés à STUDI - Reproduction interdite

Dans l'ensemble, l'imbrication est une fonctionnalité puissante de SASS qui peut grandement améliorer la lisibilité du code et faciliter la maintenance de feuilles de style volumineuses ou complexes.

Cependant, il convient de noter que l'imbrication peut également rendre le code plus difficile à lire si elle est utilisée de manière excessive ou inappropriée. Comme pour toute technique de codage, il est important d'utiliser l'imbrication judicieusement et de maintenir une hiérarchie claire des styles.

Les mixins

Les mixins sont une autre fonctionnalité puissante de SASS qui permet de gagner du temps et de réduire la duplication du code. Un mixin est un groupe de déclarations CSS qui peuvent être utilisées dans votre feuille de style. Elles sont définies à l'aide de la directive @mixin et peuvent inclure toutes les propriétés CSS valides.

L'un des principaux avantages des mixins est qu'ils permettent d'encapsuler des styles CSS complexes dans des blocs de code réutilisables. Par exemple, vous pouvez définir un mixin pour un style de bouton personnalisé qui inclut des effets de survol, des dégradés et d'autres propriétés CSS. En utilisant ce mixin dans l'ensemble de votre feuille de style, vous pouvez vous assurer que tous vos boutons ont un style cohérent, sans avoir à dupliquer le même code à plusieurs reprises.

Un autre avantage de l'utilisation des mixins est qu'ils peuvent rendre votre code plus modulaire et plus facile à maintenir. Si vous devez mettre à jour le style d'un élément particulier, il vous suffit de mettre à jour la définition du mixin plutôt que de devoir rechercher chaque instance de ce style dans l'ensemble de votre feuille de style.

Exemple

Voici un exemple de définition et d'utilisation d'un mixin dans SASS :

```scss
/* Définir un mixin */
@mixin button-style {
  display: inline-block;
  padding: 10px 20px;
  font-size: 16px;
  color: white;
  background-color: #1E88E5;
  border-radius: 5px;

  &:hover {
    background-color: #0D47A1;
  }
}

/* Utiliser le mixin */
button {
  @include button-style;
}

a.button {
  @include button-style;
  text-decoration: none;
}
```

Dans cet exemple, nous définissons un mixin appelé button-style qui définit un certain nombre de propriétés CSS pour un élément bouton. Nous utilisons ensuite la directive @include pour inclure ce mixin dans deux sélecteurs différents : button et a.button. Cela nous permet de réutiliser le même ensemble de propriétés CSS pour les éléments de type bouton et les éléments d'ancrage de la classe button.

Tous droits réservés à STUDI - Reproduction interdite

En utilisant des mixins de ce type, nous pouvons écrire un code plus concis et plus facile à maintenir, et réduire le nombre de répétitions dans nos feuilles de style.

À noter que dans l'exemple, on utilise aussi de l'imbrication avec le &:hover.

Exemple

```
button, a.button {
  display: inline-block;
  padding: 10px 20px;
  font-size: 16px;
  color: white;
  background-color: #1E88E5;
  border-radius: 5px;
}

button:hover {
  background-color: #0D47A1;
}

a.button:hover {
  background-color: #0D47A1;
  text-decoration: none;
}
```

Dans cet exemple, nous avons défini des styles pour les éléments button et a.button, et nous avons dupliqué le code de l'effet de survol pour chacun d'entre eux. Si nous décidons de modifier le style de nos boutons ultérieurement, nous devrons mettre à jour la définition du style à plusieurs endroits.

En utilisant les mixins dans SASS, nous pouvons définir des blocs de code CSS réutilisables et les inclure là où nous en avons besoin, ce qui rend notre code plus modulaire et plus facile à maintenir au fil du temps.

Dans cette vidéo, nous allons voir comment utiliser codepen.io pour utiliser SASS sans avoir à installer quoi que ce soit.

Maintenant que nous avons couvert les principales fonctions de SASS, regardons de plus près comment nous pouvons intégrer SASS dans notre flux de développement et tirer le meilleur parti de ses puissantes fonctionnalités. Dans la deuxième partie, nous explorerons certains outils et ressources essentiels pour travailler avec SASS, notamment les compilateurs, les éditeurs de texte et les ressources en ligne, pour l'apprentissage et le dépannage.

B. Exercice : Quiz

[solution n°1 p. 27]

Question 1

Quelle est la principale différence entre SASS et les feuilles de style CSS ordinaires ?

- SASS est une syntaxe alternative pour l'écriture de CSS
- SASS ajoute des fonctionnalités aux feuilles de style CSS ordinaires, telles que les variables et l'imbrication
- SASS ne fonctionne que dans certains navigateurs

Tous droits réservés à STUDI - Reproduction interdite

Question 2

Lequel des points suivants est un avantage de l'utilisation des variables SASS ?

- Cela vous permet d'écrire moins de code
- Cela vous permet d'augmenter la spécificité de vos règles CSS
- Cela vous permet d'éviter les conflits de noms dans votre CSS

Question 3

Quelle est la syntaxe pour définir une variable SASS ?

- #primary-color : #f00 ;
- $primary-color = #f00 ;
- $primary-color : #f00 ;

Question 4

Comment l'imbrication peut-elle être utilisée dans SASS pour structurer le code plus efficacement ?

- En rendant votre code plus difficile à lire
- En ajoutant des niveaux de complexité à votre code
- En vous permettant de regrouper des styles au sein d'un sélecteur parent

Question 5

Lequel des extraits de code suivants utilise correctement un mixin SASS pour définir un ensemble de styles réutilisables ?

```
.flexbox display: justify-      align-      } .container @include   }
{       flex;    content:       items:        {          flexbox;
                 center;        center;
```

```
@mixin flexbox display:     justify-content:      align-items:      }
{              flex;        center;               center;
```

```
@media  only  screen                          justify-     align-   }
and     (max-width: .container display: content:     items:     }
768px) {            {          flex;    center;       center;
```

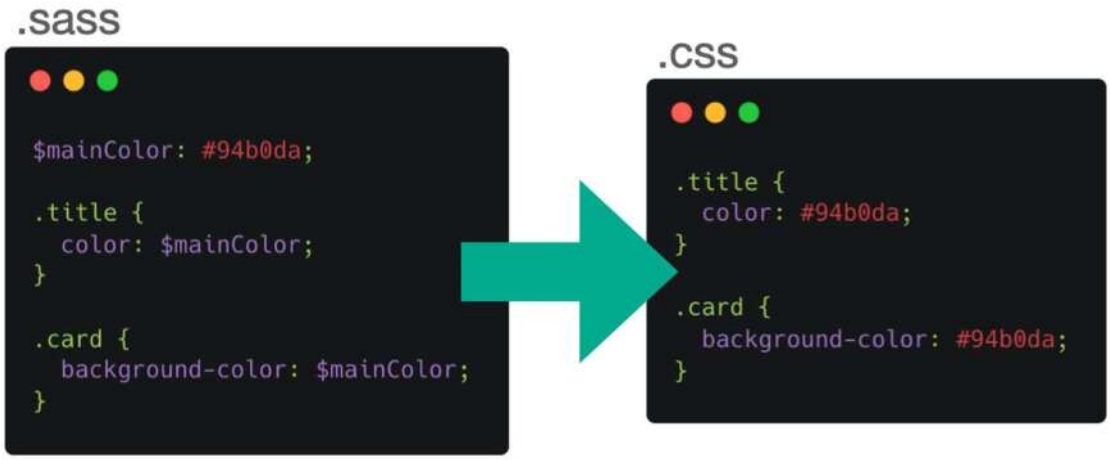
Tous droits réservés à STUDI - Reproduction interdite

III Techniques avancées SASS : optimiser votre développement CSS

A. Techniques avancées SASS : optimiser votre développement CSS

1. Compiler du code SASS en CSS

Comme nous l'avons dit précédemment, le code SASS doit être compilé en CSS pour être interprété par les navigateurs Web. Voyons un exemple :



Exemple

Exemple de compilation de code SASS en CSS.

Le code SASS suivant :

```
$primary-color: #007bff;

@mixin center {
  display: flex;
  justify-content: center;
  align-items: center;
}

.hero {
  background-color: $primary-color;
  color: white;
  padding: 50px;

  h1 {
    font-size: 48px;
    margin-bottom: 20px;
  }

  p {
    font-size: 24px;
    @include center;
  }
}
```


Tous droits réservés à STUDI - Reproduction interdite

Il est ensuite compilé en CSS comme suit :

```css
.hero {
  background-color: #007bff;
  color: white;
  padding: 50px;
}

.hero h1 {
  font-size: 48px;
  margin-bottom: 20px;
}

.hero p {
  font-size: 24px;
  display: flex;
  justify-content: center;
  align-items: center;
}
```

Il existe différentes manières de compiler le code SASS en CSS. L'une d'entre elles consiste à utiliser un outil d'Interface de Ligne de Commande (CLI) tel que Sass ou node-sass. Cette méthode oblige les développeurs à installer le compilateur et à le lancer manuellement chaque fois qu'ils souhaitent convertir leur code SASS en CSS. Une autre option consiste à utiliser un outil de construction comme Grunt ou Gulp pour automatiser le processus de compilation.

Dans ce cours, nous utiliserons le CLI, mais si vous comptez utiliser SASS dans un cadre plus professionel, vous devriez vous intéresser à Grunt ou à Gulp pour automatiser toute la compilation.

Méthode

Voici les étapes de la compilation du code SASS en CSS :

1. Écrire le code SASS en utilisant une extension de fichier .scss.
2. Compiler le code SASS en CSS à l'aide d'un compilateur ou d'un outil de construction.
3. Enregistrer le code CSS compilé dans un fichier .css distinct.
4. Lier le fichier CSS compilé à votre fichier HTML à l'aide de la balise <link>.

La vidéo suivante montre comment incorporer SASS dans son projet web et le compiler en CSS.

Fichier .map.css

Az Définition

Lorsqu'un fichier SASS est compilé, il génère un fichier CSS et un fichier source avec une extension .map. Le fichier source (.map.css) contient des informations sur le code SASS d'origine et sa relation avec le code CSS compilé. L'objectif de ce fichier est d'aider les développeurs à déboguer le code CSS, en leur permettant de remonter jusqu'à son code source d'origine. C'est un fichier JSON qui contient des correspondances entre le code CSS et le code SASS d'origine, y compris le nom du fichier d'origine, les numéros de ligne et les numéros de colonne. Il est utilisé par les outils de développement, tels que les consoles de développement des navigateurs, pour relier le code CSS compilé au code SASS d'origine. Il est ainsi plus facile d'identifier l'endroit où une règle de style particulière est définie dans le code SASS, lors du débogage de problèmes dans le code CSS.

Tous droits réservés à STUDI - Reproduction interdite

En résumé, le fichier .map généré par le compilateur SASS est un outil de débogage précieux qui aide les développeurs à retracer le code CSS compilé jusqu'au code SASS d'origine, ce qui facilite l'identification et la résolution des problèmes.

⊕ Complément

Pour aller encore plus loin dans le développement SASS sur Visual Studio Code (un éditeur de code), il est possible d'installer des extensions qui facilitent le développement.

Notamment Live Sass Compiler, qui compile les fichiers Sass ou Scss en CSS en temps réel et met à jour le CSS compilé dans le navigateur, sans qu'il soit nécessaire d'enregistrer manuellement le fichier ou de recharger la page. Il constitue un moyen pratique et efficace d'écrire et de tester du code Sass ou Scss, sans avoir à utiliser des compilateurs externes ou des outils de ligne de commande.

Il y aussi l'extension Sass. Cette extension ajoute un support linguistique pour les fichiers Sass (.sass) et SCSS (.scss). Elle fournit une coloration syntaxique, des extraits de code et une indentation automatique.

On retrouve aussi StyleLint, qui un linter moderne permettant de détecter les erreurs et d'appliquer des conventions cohérentes dans votre code SASS ou CSS. Il offre un large éventail de règles qui peuvent être personnalisées, pour correspondre au guide de style et aux normes de codage de votre équipe. Il fournit également des suggestions et des corrections pour les problèmes détectés, ce qui facilite le maintien de la qualité et de la cohérence du code dans un projet.

Az Définition

Un linter est un outil utilisé par les développeurs pour vérifier le code source à la recherche d'erreurs potentielles, de bugs et de violations des conventions de style ou de syntaxe. Il analyse le code pour détecter d'éventuels problèmes et fournit un retour d'information au développeur.

2. Les opérateurs mathématiques dans SASS

SASS fournit un certain nombre d'opérateurs mathématiques intégrés qui peuvent être utilisés pour effectuer des opérations arithmétiques dans votre feuille de style. Ces opérateurs peuvent être utilisés en conjonction avec les variables SASS pour créer des styles dynamiques et flexibles.

Voici quelques-uns des opérateurs mathématiques les plus couramment utilisés dans SASS :

- + (addition) : additionne deux valeurs
- - (soustraction) : soustrait une valeur d'une autre
- * (multiplication) : multiplie deux valeurs entre elles
- / (division) : divise une valeur par une autre

Tous droits réservés à STUDI - Reproduction interdite

Exemple

Voici un exemple où l'on définit la taille de police de certains éléments avec des opérateurs mathématiques.

```scss
$base-font-size: 16px;
$header-font-size: $base-font-size * 1.5;
$footer-font-size: $base-font-size * 0.8;

header {
  font-size: $header-font-size;
}

footer {
  font-size: $footer-font-size;
}
```

Dans cet exemple, nous définissons une taille de police de base de 16 px et utilisons des opérateurs mathématiques pour calculer les tailles de police de l'en-tête et du pied de page. La variable $header-font-size est calculée en multipliant la variable $base-font-size par 1,5, tandis que la variable $footer-font-size est calculée en la multipliant par 0,8. Ces valeurs sont ensuite utilisées pour définir la taille de la police pour les éléments d'en-tête et de pied de page respectivement.

3. Les boucles dans SASS

Les boucles sont une autre fonctionnalité puissante de SASS qui peut rendre votre code plus concis et plus efficace. Avec les boucles, vous pouvez écrire un bloc de code une fois, puis l'utiliser à plusieurs reprises avec des valeurs différentes, plutôt que d'écrire le même code encore et encore.

SASS propose deux types de boucles : @for et @each. La boucle @for permet d'itérer sur des nombres, par exemple pour compter 1,2,3,4,5… Tandis que la boucle @each est utilisée pour itérer sur des listes ou des cartes.

a. La boucle @for

Voici un exemple de boucle @for.

```scss
@for $i from 1 through 5 {
  .item-#{$i} {
    font-size: 16px * $i;
  }
}
```

Dans cet exemple, nous utilisons une boucle @for pour générer des styles pour cinq éléments différents. La boucle itère sur les valeurs 1 à 5, et pour chaque itération, elle génère une classe CSS avec une taille de police différente en fonction de la valeur actuelle de $i.

Lorsque ce code SASS est compilé, il génère le CSS suivant :

```css
.item-1 {
  font-size: 16px;
}
.item-2 {
  font-size: 32px;
}
.item-3 {
  font-size: 48px;
}
.item-4 {
  font-size: 64px;
}
```

Tous droits réservés à STUDI - Reproduction interdite

```
.item-5 {
  font-size: 80px;
}
```

b. La boucle @each

Voici un exemple de boucle @each.

```
$colors: (
  primary: #007bff,
  secondary: #6c757d,
  success: #28a745,
  danger: #dc3545,
  warning: #ffc107,
  info: #17a2b8
);

@each $name, $color in $colors {
  .btn-#{$name} {
    background-color: $color;
  }
}
```

Dans cet exemple, nous définissons un dictionnaire appelé $colors qui contient un ensemble de paires clé-valeur. Ensuite, nous utilisons la boucle @each pour itérer sur chaque paire clé-valeur dans la carte $colors. À chaque itération de la boucle, nous assignons la clé à la variable $name et la valeur à la variable $color.

À l'intérieur de la boucle, nous utilisons l'interpolation de chaînes pour générer une classe CSS appelée .btn-{nom-couleur}, où nom-couleur est le nom de chaque couleur dans la table $colors. Enfin, nous définissons la propriété background-color de chaque classe CSS sur la valeur de la couleur correspondante.

4. Les fonctions dans SASS

Les fonctions sont une autre fonctionnalité puissante de SASS qui vous permet d'écrire des morceaux de code réutilisables. Elles fonctionnent de la même manière que les fonctions dans les langages de programmation comme JavaScript, vous permettant d'effectuer des calculs, de manipuler des données, etc.

Dans SASS, vous pouvez définir vos propres fonctions à l'aide de la directive @function. Les fonctions peuvent prendre des arguments, tout comme les fonctions classiques, et renvoyer une valeur.

Exemple

Voici un exemple de fonction dans SASS :

```
@function double($number) {
  @return $number * 2;
}
.my-element {
  width: double(10px);
}
```

Dans cet exemple, nous définissons une fonction appelée double qui prend un seul argument $number. La fonction multiplie ensuite le nombre par 2 et renvoie le résultat. Dans le sélecteur .my-element, nous utilisons la fonction double pour fixer la largeur de l'élément à 20 px.

Les fonctions peuvent être extrêmement utiles pour effectuer des calculs ou d'autres opérations qui se répètent dans votre base de code. Elles peuvent également vous aider à écrire un code plus concis et plus facile à maintenir.

Tous droits réservés à STUDI - Reproduction interdite

Remarque

SASS est livré avec plusieurs fonctions intégrées que vous pouvez utiliser dès le départ. Il s'agit notamment de fonctions permettant de manipuler les couleurs, les chaînes de caractères, les listes, etc. Voici quelques exemples :

- lighten($color, $amount) : éclaircit la couleur donnée de la quantité spécifiée (en pourcentage).
- darken($color, $amount) : assombrit la couleur donnée de la quantité spécifiée (en pourcentage).
- percentage($value) : convertit la valeur donnée en pourcentage.
- join($list1, $list2, $separator) : joint deux listes ensemble, éventuellement avec un séparateur spécifié.
- color.adjust($color, $hue: $amount) est une fonction qui permet de changer la valeur d'une couleur.
- grayscale($color) est une fonction qui permet de convertir une couleur en niveaux de gris.
- invert($color) est une fonction qui permet d'inverser les couleurs d'un élément.

En utilisant des fonctions dans votre code SASS, vous pouvez simplifier vos feuilles de style et les rendre plus flexibles et réutilisables.

Petits conseils de développement

Complément

Lorsque vous développez avec SASS, il y a plusieurs astuces que vous pouvez suivre pour optimiser votre flux de travail et éviter les pièges les plus courants. Voici quelques suggestions :

- **Organisez votre code SASS :** utilisez des noms de variables significatives, une indentation cohérente et des commentaires appropriés pour rendre votre code plus lisible et plus facile à maintenir.
- **Utilisez des partiels :** divisez votre code en morceaux modulaires et importez-les en fonction de vos besoins, plutôt que d'écrire un seul gros fichier. Cela permet de réduire la taille de votre CSS compilé et de le rendre plus facile à gérer.
- **Minimisez l'imbrication :** si l'imbrication peut être utile pour organiser votre code, elle peut également conduire à des sélecteurs trop spécifiques et à des feuilles de style CSS trop volumineuses. Essayez de limiter l'imbrication à trois niveaux ou moins, et envisagez d'utiliser des conventions de nommage pour que vos sélecteurs restent concis.
- **Évitez d'utiliser trop de mixins :** si les mixins peuvent être un outil puissant, leur utilisation excessive peut entraîner une duplication du code et une diminution de la maintenabilité. Utilisez les mixins avec parcimonie et uniquement lorsqu'ils apportent une valeur significative.
- **Tirez parti des fonctions SASS :** SASS comprend de nombreuses fonctions intégrées qui peuvent vous aider à simplifier votre code et à le rendre plus efficace. Par exemple, vous pouvez utiliser les fonctions lighten() et darken() pour ajuster la luminosité d'une couleur, ou la fonction percentage() pour convertir une valeur décimale en pourcentage.

En suivant ces conseils, vous pouvez rendre votre processus de développement SASS plus efficace et produire un code plus facile à maintenir.

Tous droits réservés à STUDI - Reproduction interdite

B. Exercice : Quiz

[solution n°2 p. 29]

Question 1

Compte tenu de ce CSS compilé :

```css
.btn-primary {
  background-color: #007bff;
  border-color: #007bff;
}

.btn-secondary {
  background-color: #6c757d;
  border-color: #6c757d;
}

.btn-success {
  background-color: #28a745;
  border-color: #28a745;
}
```

Quel extrait de code SASS aurait pu être utilisé pour générer ce CSS ?

```
@for $i    $btn-                                    .btn-#
from  1 color:    @if $btn-    }  @if $btn-    }  {nth($i)} background- border- }
through #007bff;  $i  color:      $i  color:      {         color:      color:
3 {               ==  #6c757d;    ==  #28a745;             $btn-color; $btn-
                  2 {             3 {                                  color;
```

```
$colors:                              );  @each     .btn-#                    }
(        primary: secondary: success:     $color, {$color} background- border- }
         #007bff, #6c757d,   #28a745      $value  {        color:      color:
                                          in              $value;     $value;
                                          $colors
                                          {
```

```
/*  Ceci $colors:  @for $i from 1    .btn-#                  border-   }
est  une #007bff   through         {nth($i)} background-  color:       }
liste de #6c757d   length($colors) {         color:       nth($colors,
couleurs #28a745;  {                         nth($colors, $i);
*/                                           $i);
```

Question 2

Quel sera le CSS compilé de ce morceau de code :

```scss
$color: #FF5733;

@mixin mystery($size, $color) {
  $calculation: $size * 2;
  $result: darken($color, 20%);
  width: $calculation + px;
  background-color: $result;
}

.container {
  @include mystery(50, $color);
}
```


Tous droits réservés à STUDI - Reproduction interdite

```
.container {   width: 100px;   background-color: #cc2400; }
```

```
.container {   width: 50px;   background-color: #cc2400; }
```

```
.container {   width: 100px;   background-color: #ffeeff; }
```

Question 3

Laquelle des fonctions SASS suivantes peut être utilisée pour modifier la teinte d'une couleur ?

- mix()
- grayscale()
- color.adjust()

Question 4

Notre code écrit en SASS est directement utilisable par le navigateur.

- Vrai
- Faux

Question 5

Quelle est la différence entre une boucle @for et une boucle @each en SASS ?

- La boucle @for est utilisée pour itérer sur une plage numérique, alors que la boucle @each est utilisée pour itérer sur une liste de valeurs
- La boucle @each est utilisée pour itérer sur une plage numérique, alors que la boucle @for est utilisée pour itérer sur une liste de valeurs
- Les boucles @for et @each sont interchangeables et peuvent être utilisées pour tout type d'itération

Tous droits réservés à STUDI - Reproduction interdite

IV Des ressources adéquates pour un développement efficace du SASS

A. Des ressources adéquates pour un développement efficace du SASS

1. Vue d'ensemble des frameworks et bibliothèques SASS populaires qui peuvent être utilisés pour optimiser le développement

Les frameworks et bibliothèques SASS sont des collections de code SASS pré-écrites qui aident à simplifier le processus de développement. Voici quelques frameworks et bibliothèques SASS populaires que les développeurs utilisent couramment :

- **Bootstrap :** Bootstrap est un framework HTML, CSS et JS populaire qui contient une variété de composants et d'utilitaires pré-écrits. La version SASS de Bootstrap offre une flexibilité et des options de personnalisation supplémentaires.
- **Foundation :** Foundation est un autre framework HTML, CSS et JS populaire qui offre une variété de composants et d'utilitaires pré-écrits. Il est conçu pour être réactif et adapté aux mobiles.
- **Bourbon :** Bourbon est une bibliothèque SASS légère qui fournit un ensemble de mixins et de fonctions à l'usage des développeurs. C'est un outil simple et flexible qui peut aider à accélérer le développement.
- **Compass :** Compass est un framework SASS complet qui comprend de nombreuses fonctionnalités supplémentaires par rapport à SASS, telles qu'un générateur de sprites et des mixins pour la compatibilité entre les navigateurs. Compass comprend également une gamme de styles et de modèles prédéfinis, ainsi qu'une architecture modulaire qui permet aux développeurs de personnaliser et d'étendre le framework en fonction de leurs besoins.

Il existe également de nombreuses bibliothèques disponibles pour SASS, telles que Susy pour les systèmes de grille, Breakpoint pour le responsive design, et Singularity pour la mise en page et les systèmes de grille. Ces bibliothèques peuvent être ajoutées à un projet SASS à l'aide d'un gestionnaire de paquets comme npm, et peuvent considérablement accélérer le développement en fournissant du code et des fonctionnalités prêtes à l'emploi.

Dans l'ensemble, l'utilisation de SASS en combinaison avec des frameworks et des bibliothèques populaires peut considérablement améliorer la productivité et réduire le temps et les efforts nécessaires à la construction ainsi qu'à la maintenance de feuilles de style complexes.

2. La documentation de SASS

La documentation et les ressources SASS jouent un rôle crucial dans l'adoption et le succès du préprocesseur SASS. La disponibilité d'une documentation et de ressources complètes aide les développeurs à apprendre et à adopter SASS plus facilement, et leur permet également de travailler plus facilement avec le préprocesseur dans leurs projets.

La documentation SASS fournit des informations complètes sur la syntaxe, les caractéristiques et les fonctions du préprocesseur. Elle comprend des exemples, des tutoriels et des références pour aider les développeurs à apprendre et à utiliser SASS de manière efficace. La documentation est constamment mise à jour pour suivre les derniers développements du préprocesseur.

En plus de la documentation officielle, il existe également de nombreuses communautés et tutoriels en ligne qui offrent une assistance et des ressources supplémentaires aux développeurs. Ces ressources comprennent notamment des forums, des blogs, des podcasts et des vidéos. Elles constituent une plateforme permettant aux développeurs de se connecter, de partager leurs connaissances et de collaborer à des projets liés à SASS.

Tous droits réservés à STUDI - Reproduction interdite

Complément

Voici quelques exemples de ressources SASS populaires :

- Le site officiel de SASS[1] fournit une documentation complète, y compris des guides, des tutoriels et une référence de toutes les fonctions et caractéristiques de SASS.
- La communauté SASS sur Reddit[2] est un endroit idéal pour poser des questions, partager des idées et se connecter avec d'autres utilisateurs de SASS.
- La communauté SASS sur GitHub[3] est également active. Ce git couvre les nouvelles et les mises à jour liées à SASS, y compris les nouvelles fonctionnalités, les outils et les frameworks.
- CodePen[4] est une communauté en ligne populaire où les utilisateurs peuvent partager et découvrir des extraits de code, y compris des exemples de SASS.

Ces ressources peuvent s'avérer extrêmement utiles pour apprendre SASS, se tenir au courant des derniers développements et se connecter avec d'autres utilisateurs de SASS.

L'accès à ces ressources est particulièrement important pour les développeurs qui commencent à utiliser SASS, car ils peuvent avoir besoin de conseils et de soutien supplémentaires pour se familiariser avec le préprocesseur. En tirant parti des ressources disponibles, les développeurs peuvent apprendre plus rapidement, travailler plus efficacement et, en fin de compte, produire un code de meilleure qualité.

Complément

Le développement de SASS peut être considérablement amélioré avec l'aide de divers outils et plugins. En voici quelques-uns recommandés :

- **Éditeurs de code :** les éditeurs de code comme Visual Studio Code, Atom, et Sublime Text ont un support intégré pour la mise en évidence de la syntaxe de SASS, le linting, et le débogage. Ils proposent également des extensions qui peuvent améliorer votre expérience de développement avec des fonctionnalités telles que les extraits de code, l'autocomplétion et la prévisualisation en direct.
- **Préprocesseurs CSS :** SASS est déjà un préprocesseur CSS, mais il existe d'autres préprocesseurs comme Less et Stylus qui offrent des fonctionnalités similaires. Le choix du bon préprocesseur pour votre projet peut dépendre de facteurs tels que la préférence syntaxique, l'intégration avec des outils existants et le soutien de la communauté.
- **Extensions de navigateur :** les extensions de navigateur telles que LiveReload, CodeKit et Prepros peuvent automatiser le processus de compilation du code SASS en CSS, le rechargement du navigateur et l'optimisation du résultat final. Elles offrent également des fonctionnalités telles que le signalement des erreurs, la minification et le mappage des sources.

1. https://sass-lang.com/
2. https://www.reddit.com/r/sass/
3. https://github.com/rstudio/sass/blob/main/NEWS.md
4. https://codepen.io/

Tous droits réservés à STUDI - Reproduction interdite

- **Outils de build :** les outils de build tels que Gulp, Grunt et Webpack permettent d'automatiser des tâches telles que la compilation du code SASS, l'optimisation des images et le regroupement des fichiers JavaScript. Ils offrent un large éventail de plugins et de configurations qui peuvent être personnalisés pour répondre aux besoins de votre projet.

B. Exercice : Quiz

[solution n°3 p. 31]

Question 1

Quels sont les frameworks et bibliothèques SASS les plus populaires qui peuvent être utilisés pour rationaliser le développement ?

- React et Angular
- Bootstrap et Foundation
- Vue.js et Ember.js

Question 2

Quels sont les rôles de la documentation et des ressources SASS, y compris les communautés en ligne et les tutoriels ?

- A Ils fournissent des lignes directrices pour une syntaxe correcte et les meilleures pratiques
- B Ils offrent des solutions aux problèmes et défis courants
- C Ils fournissent une assistance en temps réel aux développeurs

Question 3

Quelles sont les extensions de navigateur qui peuvent améliorer le développement SASS ?

- Visual Studio Code et Sublime Text
- LiveReload, CodeKit et Prepros
- CodePen et CSS Grid Generator

Question 4

Quel éditeur de code n'est PAS recommandé pour le développement de SASS ?

- Visual Studio Code
- Sublime Text

Tous droits réservés à STUDI - Reproduction interdite

Atom

Notepad

Question 5

Comment les extensions de navigateur comme LiveReload, CodeKit et Prepros peuvent-elles aider au développement de SASS ?

Elles fournissent des éditeurs de code avancés pour écrire du code SASS

Elles permettent la compilation automatique du code SASS en CSS et le rechargement du navigateur

Elles offrent des communautés en ligne pour partager des extraits de code SASS

Tous droits réservés à STUDI - Reproduction interdite

V Synthèse

A. Flashcards

SASS

Préprocesseur CSS pour un code plus modulaire et efficace.

Variables SASS

Simplifient le code CSS et facilitent sa maintenance.

Mixins

Encapsulent des styles CSS complexes pour réutilisation.

Imbrication SASS

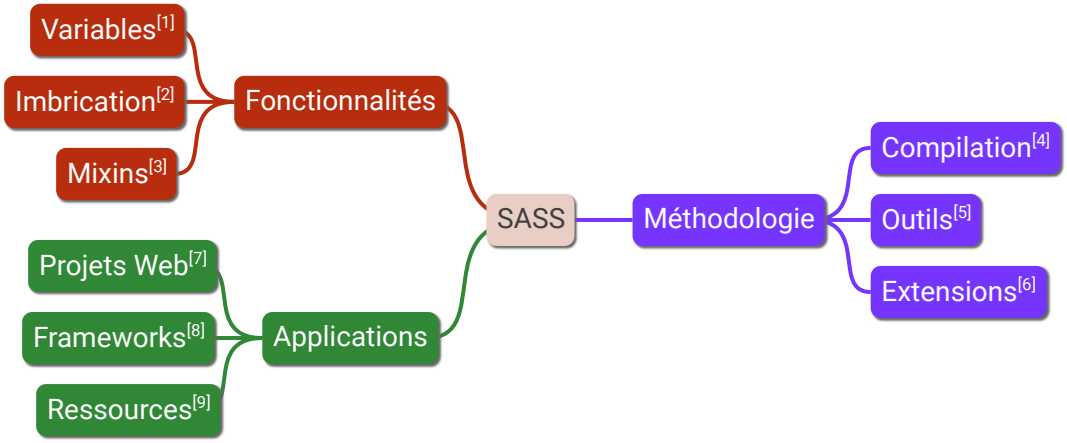
Regroupe les styles apparentés pour une meilleure lisibilité.

Compilation SASS

Convertit le code SASS en CSS pour les navigateurs.

La théorie en bref

SASS est un **préprocesseur CSS** qui simplifie l'écriture et la gestion du code CSS. Il offre des fonctionnalités comme les **variables**, l'**imbrication** et les **mixins** pour créer un code plus modulaire et efficace. Le code SASS doit être **compilé** en CSS pour être interprété par les navigateurs. Des outils comme **Grunt** ou **Gulp** peuvent automatiser ce processus. L'utilisation de **frameworks** et de **ressources** en ligne peut améliorer la productivité.



1. Simplifient le code et facilitent la maintenance.
2. Regroupe les styles apparentés pour une meilleure lisibilité.
3. Encapsulent des styles CSS complexes pour réutilisation.
4. Convertit le code SASS en CSS interprétable par les navigateurs.
5. Utilisation de CLI, Grunt ou Gulp pour automatiser la compilation.
6. Live Sass Compiler, StyleLint pour faciliter le développement.
7. Utilisation dans des projets pour améliorer la maintenabilité du CSS.
8. Intégration avec Bootstrap, Foundation pour accélérer le développement.
9. Documentation, communautés en ligne pour l'apprentissage et le dépannage.

SASS

Tous droits réservés à STUDI - Reproduction interdite

Les conseils pro

Conseil

Comment utiliser efficacement les variables SASS ?

- Définir des noms de variables clairs et significatifs.
- Utiliser les variables pour les valeurs répétées.
- Mettre à jour les variables pour changer facilement le design.

Comment optimiser le code CSS avec les mixins ?

- Créer des mixins pour les styles récurrents.
- Utiliser @include pour appliquer les mixins.
- Limiter l'utilisation excessive pour éviter la duplication.

Comment structurer le code SASS pour une meilleure lisibilité ?

- Utiliser l'imbrication avec parcimonie.
- Organiser le code en partiels modulaires.
- Maintenir une hiérarchie claire des styles.

Comment automatiser la compilation SASS en CSS ?

- Utiliser un outil CLI comme Sass ou node-sass.
- Intégrer Grunt ou Gulp pour automatiser.
- Configurer les outils pour surveiller les changements de fichiers.

Tous droits réservés à STUDI - Reproduction interdite

VI Auto-évaluation

A. Exercice

Vous êtes un développeur front-end travaillant sur un projet qui utilise SASS. Vous avez été chargé d'optimiser le code de certains boutons en utilisant les fonctionnalités avancées de SASS. Voici le code CSS de base :

```
.button-small {
  background-color: #333;
  border-radius: 5px;
  box-shadow: 1px 1px 5px #888;
  font-size: 16px;
  margin-left: 10px;

  width: 100px;
}

.button-medium {
  background-color: #333;
  border-radius: 5px;
  box-shadow: 1px 1px 5px #888;
  font-size: 16px;
  margin-left: 10px;

  width: 150px;
}

.button-large {
  background-color: #333;
  border-radius: 5px;
  box-shadow: 1px 1px 5px #888;
  font-size: 16px;
  margin-left: 10px;

  width: 200px;
}

.button-primary {
  background-color: #333;
  border-radius: 5px;
  box-shadow: 1px 1px 5px #888;
  font-size: 16px;
  margin-left: 10px;

  color: #FFF;
}

.button-secondary {
  background-color: #333;
  border-radius: 5px;
  box-shadow: 1px 1px 5px #888;
  font-size: 16px;
  margin-left: 10px;

  color: #999;
}
```


Tous droits réservés à STUDI - Reproduction interdite

Question 1

[solution n°4 p. 33]

À l'aide d'un mixin, optimisez la partie commune à tous les boutons.

Question 2

[solution n°5 p. 34]

Écrivez une fonction pour refactoriser les couleurs dans .button-primary et .button-secondary; et écrivez une boucle pour refactoriser les largeurs .button-small, .button-medium, .button-large.

B. Test

Exercice 1 : Quiz

[solution n°6 p. 34]

Question 1

Lequel des éléments suivants n'est pas un avantage de l'utilisation de SASS ?

- Il permet d'utiliser des variables, des mixins et des fonctions
- Il simplifie et réduit la quantité de code CSS nécessaire
- Il est orienté objets

Question 2

Si l'on définit $color : #f05731, lequel de ces mixins change la couleur du background en #ae2d0d ?

- `@mixin bg-color($color) {  background-color: lighten($color, 20%); }`
- `@mixin bg-color($color) {  background-color: darken($color, 20%); }`
- `@mixin bg-color($color) {  background-color: $color; }`

Question 3

Que fait la fonction suivante ?

```
@function mystery($height, $ratio) {
  $result: $height * $ratio;
  @return #{$result}px;
}
```

- Elle calcule la hauteur d'un élément, connaissant sa largeur et son ratio
- Elle calcule le ratio d'un élément, connaissant sa hauteur et sa largeur
- Elle calcule la largeur d'un élément, connaissant sa hauteur et son ratio

Question 4

Laquelle des affirmations suivantes concernant les boucles dans SASS est vraie ?

- Les boucles SASS ne sont utiles que pour effectuer des calculs

Tous droits réservés à STUDI - Reproduction interdite

Les boucles SASS ne sont utilisées que pour itérer sur des listes

Les boucles SASS peuvent être utilisées pour itérer sur n'importe quel type de valeur

Question 5

Lequel des éléments suivants est une fonction intégrée dans SASS ?

@fonction

@each

invert()

$variable

Tous droits réservés à STUDI - Reproduction interdite

Solutions des exercices

Solution n°1

[exercice p. 8]

Question 1

Quelle est la principale différence entre SASS et les feuilles de style CSS ordinaires ?

SASS est une syntaxe alternative pour l'écriture de CSS

✓ SASS ajoute des fonctionnalités aux feuilles de style CSS ordinaires, telles que les variables et l'imbrication

SASS ne fonctionne que dans certains navigateurs

Bien que SASS utilise une syntaxe alternative, sa principale différence est qu'il ajoute des fonctionnalités au CSS normal, telles que les variables et l'imbrication. L'option A est partiellement correcte, mais n'englobe pas toutes les différences principales. L'option C est entièrement fausse, SASS fonctionne partout.

Question 2

Lequel des points suivants est un avantage de l'utilisation des variables SASS ?

✓ Cela vous permet d'écrire moins de code

Cela vous permet d'augmenter la spécificité de vos règles CSS

Cela vous permet d'éviter les conflits de noms dans votre CSS

Les variables SASS vous permettent de définir une valeur une seule fois et de la réutiliser dans votre code, ce qui peut réduire considérablement la quantité de code que vous devez écrire. L'option B est incorrecte, car les variables n'affectent pas la spécificité des règles CSS, et l'option C est elle aussi incorrecte, car les conflits de noms en CSS sont généralement liés à la spécificité des sélecteurs, et non au nom des variables.

Question 3

Quelle est la syntaxe pour définir une variable SASS ?

#primary-color : #f00 ;

$primary-color = #f00 ;

Tous droits réservés à STUDI - Reproduction interdite

✓ $primary-color : #f00 ;

Les variables SASS sont définies à l'aide du symbole "*$*", comme le montre l'option C. L'option A n'utilise pas le bon symbole. L'option B utilise un "=" à la place d'un ":".

Question 4

Comment l'imbrication peut-elle être utilisée dans SASS pour structurer le code plus efficacement ?

En rendant votre code plus difficile à lire

En ajoutant des niveaux de complexité à votre code

✓ En vous permettant de regrouper des styles au sein d'un sélecteur parent

L'imbrication vous permet de regrouper des styles au sein d'un sélecteur parent, ce qui peut rendre votre code plus lisible et plus facile à organiser. L'option A est à l'opposé de la bonne réponse et l'option B est incorrecte, car l'imbrication n'ajoute pas intrinsèquement plus de complexité à votre code.

Question 5

Lequel des extraits de code suivants utilise correctement un mixin SASS pour définir un ensemble de styles réutilisables ?

```
.flexbox display: justify-      align-      } .container @include   }
{       flex;    content:       items:        {          flexbox;
                 center;        center;
```

✓
```
@mixin        display:    justify-content:     align-items:     }
flexbox {     flex;       center;              center;
```

```
@media  only  screen                          justify-      align-   }
and     (max-width: .container display: content:      items:      }
768px) {            {         flex;    center;       center;
```

Le mixin flexbox est défini pour contenir un ensemble de styles réutilisables. L'option A est fausse, car on n'utilise pas le mot-clé @mixin, tandis que l'option B utilise une media query CSS au lieu d'un mixin pour appliquer les styles.

Tous droits réservés à STUDI - Reproduction interdite

Solution n°2

[exercice p. 16]

Question 1

Compte tenu de ce CSS compilé :

```
.btn-primary {
  background-color: #007bff;
  border-color: #007bff;
}

.btn-secondary {
  background-color: #6c757d;
  border-color: #6c757d;
}

.btn-success {
  background-color: #28a745;
  border-color: #28a745;
}
```

Quel extrait de code SASS aurait pu être utilisé pour générer ce CSS ?

```
@for $i    $btn-                                  .btn-#
from  1 color:    @if $btn-    }  @if $btn-    }  {nth($i)} background- borde
through #007bff;  $i  color:      $i  color:      {         color:      color
3 {               ==  #6c757d;    ==  #28a745;              $btn-color; $btn-
                  2 {             3 {                                   color
```

✓

```
$colors:                                );  @each     .btn-#
(       primary: secondary: success:        $color, {$color} background- borde
        #007bff, #6c757d,   #28a745         $value  {        color:      color
                                            in              $value;     $valu
                                            $colors
                                            {
```

```
/*  Ceci $colors:  @for $i from 1    .btn-#                  border-  }
est  une #007bff   through        {nth($i)} background-  color:       }
liste de #6c757d   length($colors) {        color:       nth($colors,
couleurs #28a745;  {                        nth($colors, $i);
*/                                          $i);
```

Les options A et C sont équivalentes. Elles ne sont pas correctes, car elles ne génèrent pas le bon nom de sélecteur CSS.

Question 2

Quel sera le CSS compilé de ce morceau de code :

```
$color: #FF5733;

@mixin mystery($size, $color) {
  $calculation: $size * 2;
  $result: darken($color, 20%);
  width: $calculation + px;
  background-color: $result;
```

Tous droits réservés à STUDI - Reproduction interdite

```
}

.container {
  @include mystery(50, $color);
}
```

✓ `.container { width: 100px; background-color: #cc2400; }`

`.container { width: 50px; background-color: #cc2400; }`

`.container { width: 100px; background-color: #ffeeff; }`

La fonction mystery prend 2 arguments : la taille, qui vaut 50 et la couleur qui #FF5733. Elle double la taille et assombrit la couleur. B est donc fausse car la taille n'est pas doublée, et C est fausse car la couleur est #ffeeff est plus claire.

Question 3

Laquelle des fonctions SASS suivantes peut être utilisée pour modifier la teinte d'une couleur ?

mix()

grayscale()

✓ color.adjust()

mix() est une fonction qui permet de mélanger deux couleurs selon un certain pourcentage. grayscale() est une fonction qui permet de convertir une couleur en niveaux de gris. color.adjust() est une fonction intégrée qui augmente ou diminue la teinte d'une couleur, ce qui en fait la bonne réponse.

Question 4

Notre code écrit en SASS est directement utilisable par le navigateur.

Vrai

✓ Faux

SASS doit d'abord être compilé en CSS avant, sinon, nos styles ne seront pas interprétés par le navigateur.

Question 5

Quelle est la différence entre une boucle @for et une boucle @each en SASS ?

Tous droits réservés à STUDI - Reproduction interdite

✓ La boucle @for est utilisée pour itérer sur une plage numérique, alors que la boucle @each est utilisée pour itérer sur une liste de valeurs

La boucle @each est utilisée pour itérer sur une plage numérique, alors que la boucle @for est utilisée pour itérer sur une liste de valeurs

Les boucles @for et @each sont interchangeables et peuvent être utilisées pour tout type d'itération

Les boucles @for et @each sont toutes deux utilisées pour l'itération dans SASS, mais elles diffèrent dans leur syntaxe et leur objectif. Une boucle @for est utilisée pour itérer sur une plage numérique, alors qu'une boucle @each est utilisée pour itérer sur une liste de valeurs. Par exemple, @for $i from 1 through 3 itère trois fois, en donnant à $i la valeur 1, puis 2, puis 3. En revanche, @each $color in red, green, blue itère sur la liste des couleurs et attribue à $color la valeur "*red*", "*green*" et "blue" respectivement. Il est important d'utiliser le bon type de boucle en fonction de la tâche à accomplir.

Solution n°3

[exercice p. 20]

Question 1

Quels sont les frameworks et bibliothèques SASS les plus populaires qui peuvent être utilisés pour rationaliser le développement ?

React et Angular

✓ Bootstrap et Foundation

Vue.js et Ember.js

L'option A est incorrecte, car React et Angular ne sont pas des frameworks SASS, mais des frameworks JavaScript qui peuvent être utilisés pour créer des applications web. L'option C est également incorrecte, car Vue.js et Ember.js sont également des frameworks JavaScript, et non des frameworks SASS. Bootstrap et Foundation, en revanche, sont des frameworks CSS populaires qui incluent des styles et des composants prédéfinis qui peuvent être facilement personnalisés avec SASS.

Question 2

Quels sont les rôles de la documentation et des ressources SASS, y compris les communautés en ligne et les tutoriels ?

A Ils fournissent des lignes directrices pour une syntaxe correcte et les meilleures pratiques

Tous droits réservés à STUDI - Reproduction interdite

B Ils offrent des solutions aux problèmes et défis courants

C Ils fournissent une assistance en temps réel aux développeurs

À chaque problème une ressource. La documentation de SASS permet de comprendre la syntaxe, des forums comme Reddit et StackOverflow aident pour résoudre des problèmes et des blogs et podcasts permettent de se tenir au courant des nouveautés de SASS. Néanmoins, il existe des problèmes qui n'ont pas été résolus, et qui nécessiteraient l'aide d'autres développeurs en temps réel.

Question 3

Quelles sont les extensions de navigateur qui peuvent améliorer le développement SASS ?

Visual Studio Code et Sublime Text

✓ LiveReload, CodeKit et Prepros

CodePen et CSS Grid Generator

Visual Studio et Sublime Text sont des éditeurs de code. CodePen et CSS Grid Generator sont des sites.

Question 4

Quel éditeur de code n'est PAS recommandé pour le développement de SASS ?

Visual Studio Code

Sublime Text

Atom

✓ Notepad

Notepad est un simple éditeur de texte et n'est pas recommandé pour le développement de SASS. Visual Studio Code, Sublime Text et Atom sont des éditeurs de code populaires qui sont recommandés pour le développement de SASS, parce qu'ils ont un support intégré pour la coloration syntaxique de SASS, l'auto-complétion et d'autres fonctionnalités utiles.

Tous droits réservés à STUDI - Reproduction interdite

Question 5

Comment les extensions de navigateur comme LiveReload, CodeKit et Prepros peuvent-elles aider au développement de SASS ?

Elles fournissent des éditeurs de code avancés pour écrire du code SASS

✓ Elles permettent la compilation automatique du code SASS en CSS et le rechargement du navigateur

Elles offrent des communautés en ligne pour partager des extraits de code SASS

L'option A est incorrecte, car les extensions de navigateur ne sont pas des éditeurs de code. Elles n'offrent pas de fonctionnalités avancées pour l'écriture de code SASS, mais elles peuvent vous aider à écrire un meilleur code en vous fournissant des rapports d'erreurs et d'autres informations en retour.

L'option C est incorrecte, car les extensions de navigateur ne sont pas des communautés en ligne. Bien qu'il existe des communautés en ligne pour le partage d'extraits de code SASS, les extensions de navigateur sont des outils autonomes qui peuvent être utilisés indépendamment de toute communauté.

[exercice p. 25]

Solution n°4

```scss
@mixin button-base {
  background-color: #333;
  border-radius: 5px;
  box-shadow: 1px 1px 5px #888;
  font-size: 16px;
  margin-left: 10px;
}

.button-small {
@include button-base;
  width: 100px;
}

.button-medium {
@include button-base;
  width: 150px;
}

.button-large {
@include button-base;
  width: 200px;
}

.button-primary {
@include button-base;
  color: #FFF;
}

.button-secondary {
@include button-base;
  color: #999;
```

Tous droits réservés à STUDI - Reproduction interdite

```
}
```

[exercice p. 25]

Solution n°5

```scss
@mixin button-base {
  background-color: #333;
  border-radius: 5px;
  box-shadow: 1px 1px 5px #888;
  font-size: 16px;
  margin-left: 10px;
}
@function button-text-color($button-type) {
  @if $button-type == 'primary' {
    @return #FFF;
  } @else if $button-type == 'secondary' {
    @return #999;
  }
}

$button-sizes: (
  small: 100px,
  medium: 150px,
  large: 200px
);

@each $size, $width in $button-sizes {
  .button-#{$size} {
    @include button-base;
    width: $width;
  }
}

.button-primary {
  @include button-base;
  color: button-text-color('primary');
}

.button-secondary {
  @include button-base;
  color: button-text-color('secondary');
}
```

Solution n°6

[exercice p. 25]

Question 1

Lequel des éléments suivants n'est pas un avantage de l'utilisation de SASS ?

Il permet d'utiliser des variables, des mixins et des fonctions

Il simplifie et réduit la quantité de code CSS nécessaire

✓ Il est orienté objets

Tous droits réservés à STUDI - Reproduction interdite

Les options A et B sont vraies, SASS permet de simplifier le CSS avec des fonctionnalités comme les mixins. Mais ce n'est pas un langage orienté objets.

Question 2

Si l'on définit $color : #f05731, lequel de ces mixins change la couleur du background en #ae2d0d ?

```
@mixin bg-color($color) {   background-color: lighten($color, 20%); }
```

✓
```
@mixin   bg-color($color)      background-color:   darken($color, }
{                          20%);
```

```
@mixin bg-color($color) {   background-color: $color; }
```

#ae2d0d est plus sombre que #f05731, le seul des 3 mixins qui assombrit une couleur est l'option B.

Question 3

Que fait la fonction suivante ?

```
1 @function mystery($height, $ratio) {
2   $result: $height * $ratio;
3   @return #{$result}px;
4 }
```

Elle calcule la hauteur d'un élément, connaissant sa largeur et son ratio

Elle calcule le ratio d'un élément, connaissant sa hauteur et sa largeur

✓ Elle calcule la largeur d'un élément, connaissant sa hauteur et son ratio

La fonction prend deux arguments : la hauteur et le ratio, en multipliant les deux on obtient la largeur.

Question 4

Laquelle des affirmations suivantes concernant les boucles dans SASS est vraie ?

Les boucles SASS ne sont utiles que pour effectuer des calculs

Les boucles SASS ne sont utilisées que pour itérer sur des listes

✓ Les boucles SASS peuvent être utilisées pour itérer sur n'importe quel type de valeur

Tous droits réservés à STUDI - Reproduction interdite

La bonne réponse est C. Les boucles SASS peuvent être utilisées pour itérer sur n'importe quel type de valeur. Les boucles SASS ne sont pas limitées aux calculs ou à l'itération de listes. Elles peuvent être utilisées pour itérer sur n'importe quel type de valeur, y compris les nombres, les chaînes de caractères, les couleurs, etc. Cette flexibilité permet d'obtenir un code SASS plus dynamique et plus efficace.

Question 5

Lequel des éléments suivants est une fonction intégrée dans SASS ?

- @fonction
- @each
- ✓ invert()
- $variable

La bonne réponse est C invert(). invert() est une fonction intégrée dans SASS qui inverse la couleur d'une valeur donnée. @function est utilisé pour définir une fonction personnalisée, @each est utilisé pour les boucles et $variable est utilisé pour déclarer une variable dans SASS.

Tous droits réservés à STUDI - Reproduction interdite